CASE STUDY

One patient. Three files. One clinical question.

How do we combine data accurately without copy-and-paste?

patients.csv

visits.csv

labs.csv

Who had a visit with an above-reference HbA1c result?

What you should be able to do

SQL fluency matters more than memorizing database brands.

01 Find

SELECT · WHERE · ORDER BY

02 Summarize

COUNT · AVG · GROUP BY · HAVING

03 Relate

INNER JOIN · LEFT JOIN

04 Transform safely

CASE · INSERT/UPDATE/DELETE

05 Reproduce

Query the same SQLite database from R

Our dataset: deliberately imperfect links

Synthetic clinical encounters; no real patient data.

32

patients

3 have no visit

59

visits

5 have no lab record

117

labs

2 missing result values

Question What should happen when a patient has no visit or a visit has no labs?

The database ecosystem

SQL is a language, not a server

Separate the query syntax, software implementation, and data model.

Language

SQL = Structured Query Language

Common verbs: SELECT, JOIN, GROUP BY

Relational DBMS

SQLite · PostgreSQL · MySQL · MariaDB · SQL Server · Oracle

Different model

MongoDB = document database

Not another SQL relational engine

Key message Transferable SQL concepts ≠ identical syntax or capabilities.

A map of major relational systems

All support SQL, but differ in deployment, features, dialects, and ecosystem.

System	Typical architecture	Illustrative fit
SQLite	Embedded; a database file	Course lab, portable apps
PostgreSQL	Open-source server	Research / multi-user
MySQL	Open-source server	Web and existing estates
MariaDB	MySQL-origin open-source server	MySQL-related environments
SQL Server	Microsoft server platform	Institutional enterprise
Oracle Database	Commercial enterprise RDBMS	Large organizational systems

MySQL and MariaDB: related, not interchangeable

MariaDB began as a fork of MySQL; both continue to evolve independently.

Shared background

Both are relational database systems with familiar SQL concepts and a common historical lineage.

Important differences

Version-specific features, storage engines, client compatibility, licensing, performance and operational choices vary.

Practical rule Do not assume a drop-in replacement without testing the application and SQL dialect.

Embedded file or database server?

Same conceptual queries; very different deployment experience.

SQLite | Embedded

Your R session ↔ single .sqlite file
No dedicated server or account
Excellent for reproducible teaching

PostgreSQL/MySQL/MariaDB/etc.

R client ↔ network ↔ database server
Credentials, users, roles, backup, governance
Supports centralized shared services

Today Each student gets the same portable SQLite database; no remote login needed.

Relational tables vs document collections

Modeling choices affect data organization and query patterns.

Relational model	Document-oriented model
Rows in tables; explicit columns	Documents in collections
Keys connect entities across tables	Related fields can be nested or referenced
JOIN is central for normalized entities	Document queries / aggregation pipelines
Good for linked, stable entities	Good for document-shaped, evolving records
Example: PostgreSQL / SQLite	Example: MongoDB

Which database should you choose?

Begin with requirements, not the popularity of a product.

Local & reproducible

SQLite: coursework, prototypes and portable data

Multi-user research

PostgreSQL: centralized, feature-rich open source

Existing infrastructure

MySQL/MariaDB: fit with supported stack

Enterprise mandate

SQL Server or Oracle: organizational fit

Nested documents

MongoDB: assess a document-oriented model

Controlled clinical data

Governance and permissions come before preference

PAUSE & DISCUSS

Check your understanding

Which statement is correct?

A

SQL and PostgreSQL are synonyms.

B

MariaDB and MySQL must behave identically.

C

SQLite is a relational database in a file.

D

MongoDB is a relational SQL server.

Answer

PAUSE & DISCUSS

Check your understanding

Which statement is correct?

A

SQL and PostgreSQL are synonyms.

B

MariaDB and MySQL must behave identically.

C

SQLite is a relational database in a file.

D

MongoDB is a relational SQL server.

Answer C. SQL is a language; SQLite is a database system.

Relational essentials

The schema is a map of relationships

Know the key linking each table before writing JOIN.

patients

patient_id PK
sex_at_birth
birth_year
zip3

visits

visit_id PK
patient_id FK
visit_date
visit_type

labs

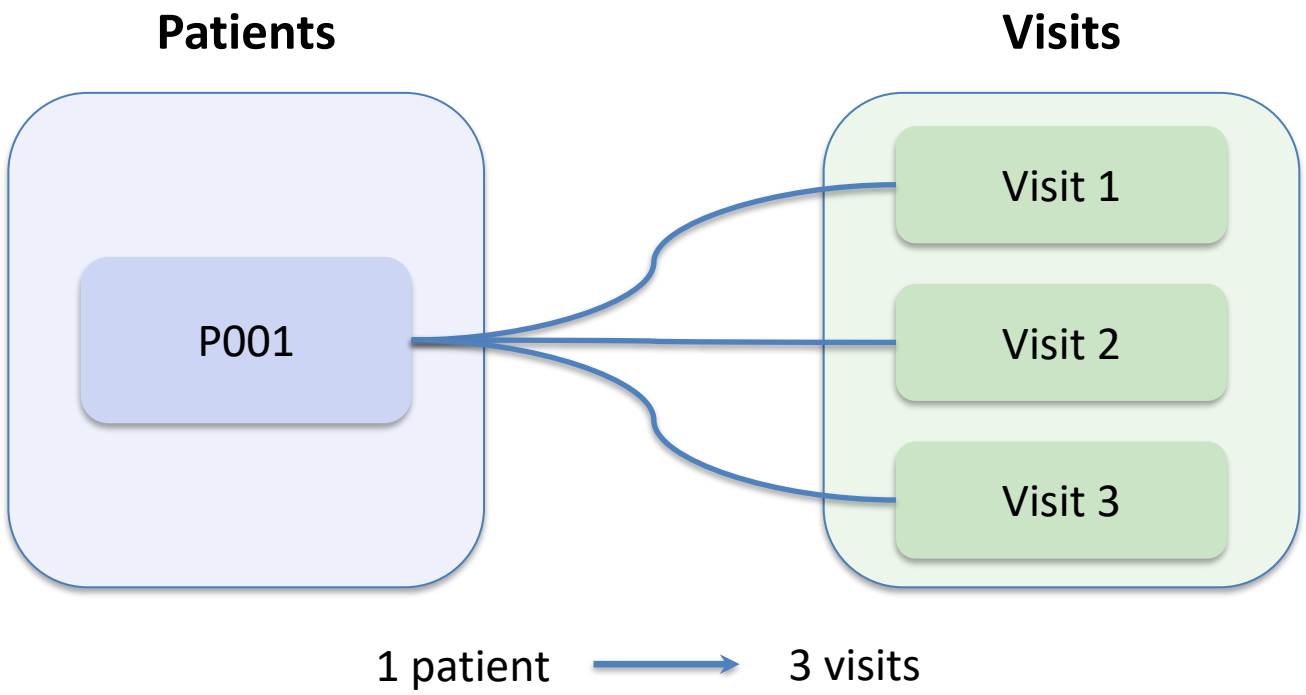
lab_id PK
visit_id FK
test_name
result_value

1 → many

1 → many

The schema is a map of relationships

Know the key linking each table before writing JOIN.



What does ONE row represent?

Always identify what one row represents. It determines how you should analyze the data

patients

One row = one patient

patient_id identifies it.

visits

One row = one visit

A patient may appear several times.

labs

One row = one lab result

A visit may have several results.

Every query Before running it: “What will one output row represent?”

Keys prevent accidental cross-products

Match tables using the relationship and not merely similar-looking columns.

SQL

```
FROM patients AS p
JOIN visits AS v
  ON p.patient_id = v.patient_id
```

Why use ON?

The join condition determines which rows represent the same linked entity. Without it, a Cartesian product may produce a huge, incorrect result.

Count check 32 × 59 = 1,888 possible cross-pairs; only 59 matches are valid.

Connect and inspect

SQLite: open a local database file

No server, account, host or port is needed for the class dataset.

Terminal / SQLite shell

```
sqlite3 Data/bioe860_week6_clinical.sqlite
.tables
.schema visits
SELECT COUNT(*) FROM visits;
.quit
```

Notice

The .tables and .schema commands are SQLite shell shortcuts.

A wrong file path may create a new empty database.

PostgreSQL: connect using psql

Server-based: the client connects to a named database with an approved account.

Terminal / psql session

```
psql -h db.example.edu -p 5432 \
  -U researcher -d research_db
\dt
\d visits
SELECT COUNT(*) FROM visits;
\q
```

What changes?

-h host • -p port

-U user • -d database

Authentication and access rights are controlled by the server.

MySQL / MariaDB: connect using a client

Similar command-line workflow; engine version and installed client determine details.

Terminal / SQL client

```
mysql -h db.example.edu -P 3306 \
  -u researcher -p research_db
-- For MariaDB, use: mariadb ...
SHOW TABLES;
DESCRIBE visits;
SELECT COUNT(*) FROM visits;
exit
```

Key idea

MySQL: mysql client

MariaDB: mariadb client

-p prompts for a password. Do not paste passwords into the command.

How do I find the tables and inspect a schema?

Start with discovery before writing joins. The exact inspection command depends on the DBMS.

SQLite shell	`.tables`	`.schema visits`
PostgreSQL / psql	`\dt`	`\d visits`
MySQL	`SHOW TABLES;`	`DESCRIBE visits;`
MariaDB	`SHOW TABLES;`	`DESCRIBE visits;`

Client-specific shortcuts are not interchangeable. For SQLite file, try: PRAGMA table_info(visits);

SQLite schema inspection with SQL

PRAGMA lets you inspect fields and key relationships without altering the table.

Useful SQLite inspection queries

```
SELECT name FROM sqlite_schema
WHERE type = 'table' ORDER BY name;

PRAGMA table_info(visits);
PRAGMA foreign_key_list(visits);

SELECT * FROM visits LIMIT 5;
```

Check before querying

What is the primary key?
Which column is a foreign key?
What does one row represent?
Could a JOIN multiply rows?

Portable discovery: INFORMATION_SCHEMA

Useful in PostgreSQL, MySQL and MariaDB; SQLite uses different introspection tools.

PostgreSQL example

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
  AND table_type = 'BASE TABLE';

SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'public'
  AND table_name = 'visits'
ORDER BY ordinal_position;
```

MySQL / MariaDB

Use:

WHERE table_schema = DATABASE()

Then filter by table_name.

SQLite: use PRAGMA.

Use the same DBI pattern across database engines

Change the driver and connection arguments—not your scientific question.

SQLite (RSQLite)

```
library(DBI)
library(RSQLite)
con <- dbConnect(
  SQLite(),
  dbname = "Data/bioe860_week6_clinical.sqlite"
)
dbListTables(con)
dbDisconnect(con)
```

PostgreSQL (RPostgres)

```
library(DBI)
library(RPostgres)
con <- dbConnect(Postgres(),
 dbname="research_db",
 host=Sys.getenv("DB_HOST"),
 user=Sys.getenv("DB_USER"),
 password=Sys.getenv("DB_PASSWORD"))
dbDisconnect(con)
```

MySQL / MariaDB: library(RMariaDB); dbConnect(MariaDB(), dbname=..., host=..., user=..., password=...); dbDisconnect(con)

Explore → query → close from R

The DBI interface lets one workflow work across multiple backends.

After opening con in R

```
dbIsValid(con)
dbListTables(con)
dbListFields(con, "visits")

dbGetQuery(con,
  "SELECT * FROM visits LIMIT 5")

dbDisconnect(con)
```

For large data

Inspect schema first.

Retrieve a small sample.

Prefer filtering + aggregation in SQL.

Close connections reliably.

Retrieve, filter, sort

Read a query from the inside out

SQL clauses build a precise question.

SQL

```
SELECT patient_id, birth_year
FROM patients
WHERE birth_year <= 1961
ORDER BY birth_year, patient_id;
```

Translated

“From patients, return these two columns for people born in 1961 or earlier, oldest first.”

SELECT: choose exactly what you need

Projection selects columns; it does not remove rows unless DISTINCT applies.

SQL

```
SELECT patient_id, sex_at_birth, birth_year
FROM patients;
```

Why not SELECT *?

Explicit columns make the analytic extract easier to interpret and more stable when the schema changes.

Think ahead Will you need one row per patient or a summary by subgroup?

WHERE: filter rows before analysis

Use comparisons, AND/OR, and parentheses to reflect the research rule.

SQL

```
SELECT visit_id, patient_id, visit_type
FROM visits
WHERE visit_type = 'Cardiology'
  AND systolic_bp >= 140;
```

Question

Which cardiology visits meet the specified example blood-pressure filter?

Practice Predict whether changing AND to OR would return more or fewer rows.

DISTINCT and ORDER BY

Deduplicate outputs and make result order explicit.

SQL

```
SELECT DISTINCT visit_type
FROM visits
ORDER BY visit_type;
```

A subtlety

ORDER BY controls presentation. It does not aggregate. DISTINCT collapses duplicate result combinations.

Do not assume A database query without ORDER BY has no guaranteed row order.

NULL is not zero and not an empty string

Missingness changes comparisons and aggregation.

SQL

```
SELECT visit_id, bmi
FROM visits
WHERE bmi IS NULL;
```

Avoid this error

WHERE bmi = NULL

SQL missing-value tests use IS NULL or IS NOT NULL.

Dataset 4 visits have a missing BMI; 2 lab results have missing values.

CHECKPOINT

One-minute SQL challenge

Write a query before looking at the solution.

List patient ID and birth year for patients born before 1970, ordered from oldest to youngest.

SQL

```
SELECT patient_id, birth_year
FROM patients
WHERE birth_year < 1970
```

Aggregate and summarize

Aggregation: rows become summaries

Functions combine multiple observations into a statistic.

COUNT(*)

All rows, including rows with NULL values

COUNT(bmi)

Rows with nonmissing BMI

AVG(bmi)

Average of available BMI values

SUM / MIN / MAX

Totals and extremes for a numeric variable

Reality check 59 visits, but COUNT(bmi) = 55.

GROUP BY answers subgroup questions

One output row per distinct grouping value.

SQL

```
SELECT visit_type,
       COUNT(*) AS n_visits,
       ROUND(AVG(bmi), 1) AS mean_bmi
FROM visits
GROUP BY visit_type;
```

Interpretation

Four output rows: Cardiology, Endocrinology, Neurology and Primary care.

WHERE filters rows; HAVING filters groups

Both are filters—but they operate at different stages.

SQL

```
SELECT visit_type, COUNT(*) AS n
FROM visits
WHERE visit_date >= '2025-04-01'
GROUP BY visit_type
HAVING COUNT(*) >= 4;
```

Order of operations

WHERE limits the input visits; GROUP BY forms groups; HAVING filters summarized groups.

CHECKPOINT

Aggregation challenge: what is being counted?

Every change in function changes the denominator.

SQL

```
SELECT test_name,
       COUNT(*) AS lab_rows,
       COUNT(result_value) AS measured,
       AVG(result_value) AS mean_value
FROM labs
GROUP BY test_name;
```

Question!

Why might lab_rows and measured differ for some test_name values?

Remember COUNT(*) ≠ COUNT(column) when that column has missing values.

JOIN related tables

INNER JOIN: retain matching records

Join patients and visits on their key.

SQL

```
SELECT p.patient_id, v.visit_id, v.visit_date
FROM patients AS p
INNER JOIN visits AS v
  ON p.patient_id = v.patient_id;
```

Expected result

59 rows.

Every visit belongs to a patient; patients with no visits are excluded.

Question What does one result row now represent? A visit (not a patient).

LEFT JOIN: preserve unmatched left rows

Find patients who have no visit at all.

SQL

```
SELECT p.patient_id
FROM patients AS p
LEFT JOIN visits AS v
  ON p.patient_id = v.patient_id
WHERE v.visit_id IS NULL;
```

Output

P030
P031
P032

3 patients without a visit.

Predict the JOIN row count

The output grain determines the number of records.

Query	Expected rows	Reason
patients INNER JOIN visits	59	One row per matched visit
patients LEFT JOIN visits	62	59 matched + 3 unmatched
visits INNER JOIN labs	117	One row per matched lab
visits LEFT JOIN labs	122	117 labs + 5 unmatched visits

Quality check More rows after JOIN can be correct; unexplained rows are a warning.

A LEFT JOIN can silently become an INNER JOIN

A WHERE condition on the right table can drop all unmatched rows.

SQL

```
SELECT p.patient_id, v.visit_type
FROM patients p
LEFT JOIN visits v
 ON p.patient_id = v.patient_id
WHERE v.visit_type = 'Cardiology';
```

Why?

Unmatched patients have NULL visit_type.

WHERE removes them.

Design question Are you filtering matches only or all left-table rows?

CHECKPOINT

JOIN challenge: visits with no lab results

Translate a missing-data question into a LEFT JOIN.

SQL

```
SELECT v.visit_id
FROM visits AS v
LEFT JOIN labs AS l
  ON v.visit_id = l.visit_id
WHERE l.lab_id IS NULL
ORDER BY v.visit_id;
```

Expected IDs

V055
V056
V057
V058
V059

Conditional logic & safe manipulation

CASE WHEN: classify without editing source data

Computed fields can make complex filters and summaries easier.

SQL

```
SELECT lab_id, result_value,
  CASE
    WHEN result_value IS NULL THEN 'Missing'
    WHEN result_value < ref_low THEN 'Below reference'
    WHEN result_value > ref_high THEN 'Above reference'
    ELSE 'Within reference'
  END AS result_flag
FROM labs;
```

Computed output only

DQL reads; DML changes stored rows

Separate safe querying from operations that modify the database.

DQL | Data query

SELECT

Read, filter, aggregate and join records.

DML | Data manipulation

INSERT · UPDATE · DELETE

Changes persist unless reverted.

Terminology Some classifications include SELECT under DML; DQL is a helpful teaching distinction.

Transaction + WHERE: make DML safer

Practice modifications only in student_notes, then roll them back.

SQL

```
BEGIN TRANSACTION;
UPDATE student_notes
SET note_text = 'Revised example'
WHERE patient_id = 'P001';
SELECT * FROM student_notes WHERE patient_id='P001';
ROLLBACK;
```

Guardrails

Work on a copy.

Use narrow WHERE.

Verify changes.

ROLLBACK preserves original state.

8

CONNECT → QUERY → COLLECT → DISCONNECT

Use SQL from R

R ↔ SQLite: the complete workflow

Use DBI (interface) + RSQLite (backend).

R

```
library(DBI)
library(RSQLite)
con <- dbConnect(SQLite(),
  dbname = "Data/bioe860_week6_clinical.sqlite")
dbListTables(con)
# Run queries, then disconnect
dbDisconnect(con)
```

No login required

Execute SQL from R with dbGetQuery()

SQL runs in SQLite; results arrive in R as a data frame.

R code with SQL

```
counts <- dbGetQuery(con, "
  SELECT visit_type, COUNT(*) AS n_visits
  FROM visits
  GROUP BY visit_type
  ORDER BY visit_type
")
print(counts)
```

Key idea

Filter and aggregate in the database; retrieve only the results you need.

Use parameters—not pasted input

Parameter placeholders make queries safer and easier to reuse.

R + SQL

```
year_limit <- 1961
result <- dbGetQuery(
  con,
  "SELECT patient_id FROM patients
   WHERE birth_year <= ?",
  params = list(year_limit)
)
```

Why?

Avoid string-concatenated input; use supported parameter binding.

Optional: dplyr can generate SQL

dbplyr bridges familiar R verbs and SQL—but read the query.

R / dbplyr

```
library(dplyr)
library(dbplyr)
query <- tbl(con, 'visits') |>
  group_by(visit_type) |>
  summarize(n = n(), .groups = 'drop')
show_query(query)
collect(query)
```

Scope note

Optional extension. Direct SQL remains core for this session.

Lab and homework

APPLY

Lab: build complexity one step at a time

All tasks use one portable SQLite file.

A Inspect the schema

Student worksheet + starter code

B Retrieve + filter

Student worksheet + starter code

C Group + aggregate

Student worksheet + starter code

D Join related tables

Student worksheet + starter code

E Find unmatched rows

Student worksheet + starter code

F CASE + safe DML

Student worksheet + starter code

G Connect from R

Student worksheet + starter code

Teaching habit Predict the output row grain and count before running the query.

ASSESS

Homework 6: independent clinical SQL queries

Seven components, 100 points, with SQL and R deliverables.

Component	Points
Retrieval / filtering	15
Aggregations	20
JOINs / grain	25
CASE WHEN	15
Safe DML	10
SQL from R	10
Interpretation	5

TAKE HOME

Five transferable habits

Core SQL will work across systems, with dialect adjustments.

1

Know one row’s meaning.

2

Use keys, not guesses.

3

Check JOIN counts.

4

Treat NULL and DML carefully.

5

Query in the database; retrieve only needed data.

References

- SQLite | sqlite.org/about.html
- PostgreSQL | postgresql.org/docs/
- MySQL | dev.mysql.com/doc/
- MariaDB | mariadb.org/documentation/
- Microsoft SQL Server | learn.microsoft.com/sql/
- MongoDB | mongodb.com/docs/
- R DBI / RSQLite | dbi.r-dbi.org / rsqlite.r-dbi.org